National Institute of Technology, Mizoram

Design and Verification of a Parameterized Counter Using Control FSM in SystemVerilog

Author:
Mohammad Rehan

Project Type:
Self-Initiated Learning Project

Tools Used:
SystemVerilog, Xilinx Vivado Simulator, FPGA Synthesis Tools

Key Concepts Demonstrated:
- Finite State Machine (FSM) Design
- Control–Datapath Architecture
- Parameterized RTL Design
- SystemVerilog Testbench Development
- Functional Simulation and Waveform Analysis
- FPGA Synthesis and Timing Analysis

Project Summary

This project presents the design and verification of a parameterized digital counter using SystemVerilog.
The design follows a modular control–datapath architecture where a

finite state machine (FSM) controls the operation of the counter Datapath.

The counter supports start/stop control, enable logic, up-count and down-count modes, and a configurable maximum count value. Functional verification was performed using a SystemVerilog testbench, and synthesis results were analyzed using FPGA development tools.

Development Environment

RTL Language: SystemVerilog
Simulation Tool: Xilinx Vivado Simulator
Design Methodology: RTL Design and Functional Verification

Year: 2026

---

1. Introduction

Digital counters are fundamental components in digital systems and are widely used in applications such as timers, frequency dividers, event counters, and embedded control systems. Counters can operate in different modes such as up-counting or down-counting and often require control logic to start, stop, or reset operations.

This project presents the **design and verification of a parameterized counter using SystemVerilog**, implemented with a **control finite state machine (FSM)** and a **separate datapath module**. The design demonstrates the concept of **modular hardware design**, where control logic and data processing logic are implemented independently.

The counter supports the following features:

- Start and stop control signals
- Enable control
- Up and down counting modes
- Parameterized counter width
- Maximum count limit control

The design is verified using a **SystemVerilog testbench**, and simulation results are observed through waveform analysis.

---

2. Objectives

The main objectives of this project are:

- To design a **parameterized digital counter** using SystemVerilog.
- To implement **control logic using a Finite State Machine (FSM)**.
- To separate the design into **control and datapath modules**.
- To implement **up-count and down-count functionality**.
- To verify the design through **simulation and waveform analysis**.

---

3. System Architecture

The overall architecture of the system consists of three main components:

1. Top Module (counter_top)
2. Control FSM (control_fsm)
3. Datapath Module (counter_datapath)

The architecture follows a **control-datapath separation design methodology**, which is widely used in hardware design to improve modularity and scalability.

3.1 System Block Diagram

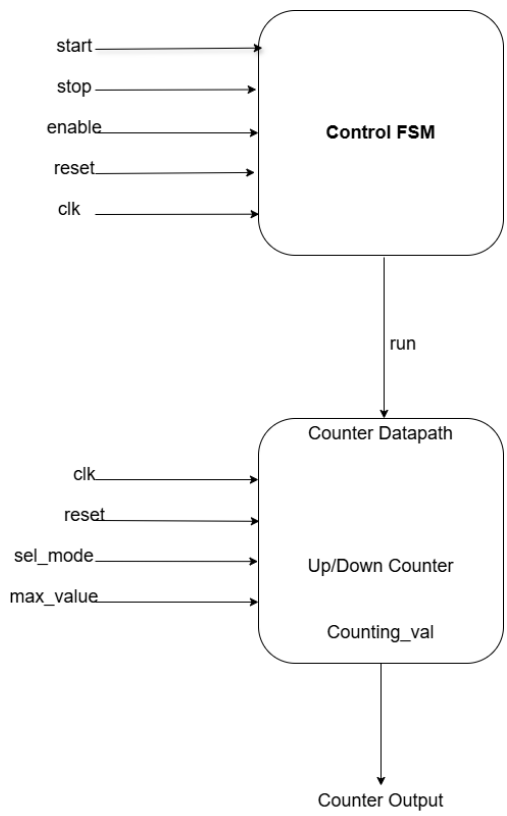

Figure 1: Block Diagram of the Parameterized Counter System

4. Module Description

4.1 Top Module

The counter_top module acts as the integration layer connecting the control FSM and datapath modules.

Inputs:

Signal	Description
clk	System clock
reset	Reset signal
start	Start counter operation
stop	Stop counter operation
enable	Enable counter
sel_mode	Mode selection (Up/Down counting)
max_value	Maximum counting value

Table 1: Input signals (Top module).

Output:

Signal	Description
counting_val	Current counter value

Table 2: Output signals (Top module).

Function

The top module instantiates:

- Control FSM to generate the run signal
- Datapath module to perform actual counting operations

4.1.1 Top Module (counter_top.sv)

```
module counter_top #(
   parameter WIDTH = 8
)(
   input  logic clk,
   input  logic reset,
   input  logic start,
   input  logic stop,
   input  logic enable,
   input  logic sel_mode,
   input  logic [WIDTH-1:0] max_value,
   output logic [WIDTH-1:0] counting_val
);
logic run;
// CONTROL
control_fsm u_ctrl (
   .clk(clk),
```

```
    .reset(reset),
    .start(start),
    .stop(stop),
    .enable(enable),
    .run(run)
);
// DATAPATH
counter_datapath #(.WIDTH(WIDTH)) u_data (
    .clk(clk),
    .reset(reset),
    .run(run),
    .direction(sel_mode),
    .max_value(max_value),
    .count(counting_val)
);
endmodule
```

4.2 Control FSM

The control FSM controls when the counter should run.

Inputs:

- start
- stop

- enable
- reset

Output:

- run

4.2.1 FSM (Finite State Machine) State Diagram

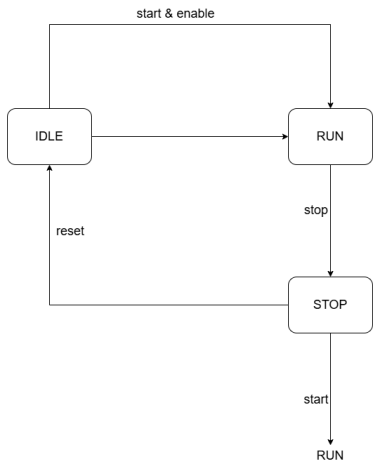

Figure 2: Finite State Machine (FSM) Diagram for Counter Control

---

4.2.2 FSM Operation

The FSM generates a run signal that controls the datapath.

Typical behavior:

Condition	**Action**
start & enable	run = 1
stop	run = 0
reset	run = 0

Table 3: FSM signals.

This ensures the datapath only counts when the system is enabled and started.

4.2.3 Control FSM Module (control_fsm.sv)

```
module control_fsm(
   input  logic clk,
   input  logic reset,
   input  logic start,
   input  logic stop,
   input  logic enable,
   output logic run
);
typedef enum logic [1:0] {
   IDLE,
   RUN,
   STOP
} state_t;
state_t state, next_state;
// State register
always_ff @(posedge clk) begin
   if(reset)
      state <= IDLE;
```

```
    else
        state <= next_state;
end
// Next state logic
always_comb begin
    next_state = state;
    case(state)
        IDLE:
            if(enable && start)
                next_state = RUN;
        RUN:
            if(stop)
                next_state = STOP;
        STOP:
            if(start)
                next_state = RUN;
    endcase
end
// Output logic
assign run = (state == RUN);
endmodule
```

4.3 Datapath Module

The datapath module implements the actual counter logic.

Inputs:

Signal	Description
clk	System clock
reset	Reset signal
run	Run signal from FSM
direction	Up/Down counting control
max_value	Maximum count limit

Table 4: Datapath inputs.

Output:

Signal	Description
count	Current counter value

Table 5: Datapath outputs.

Counter Modes

Up Counting Mode (direction = 1)
The counter increments until it reaches max_value.

Down Counting Mode (direction = 0)
The counter decrements until it reaches zero.

4.3.1 Counter Datapath Module (counter_datapath.sv)

```
module counter_datapath #(
   parameter WIDTH = 8
)(
   input  logic clk,
   input  logic reset,
   input  logic run,
   input  logic direction,   // 1=UP
   input  logic [WIDTH-1:0] max_value,
   output logic [WIDTH-1:0] count
);
always_ff @(posedge clk) begin
   if(reset)
      count <= '0;
   else if(run) begin
      if(direction) begin
         count <= (count == max_value)
                 ? '0
                 : count + 1;
      end
```

```
      else begin
         count <= (count == 0)
                ? max_value
                : count - 1;
      end
   end
end
endmodule
```

---

5. Testbench Design

A **SystemVerilog testbench** was developed to verify the functionality of the counter.

The testbench includes:

- Clock generation
- Reset sequence
- Start and stop control
- Mode switching
- Monitoring of counter output

5.1 Clock Generation

A periodic clock signal is generated using:

```
always #5 clk = ~clk;
```

This produces a 10 ns clock period.

5.2 Test Tasks

Three reusable tasks are created:

Reset Task

Initializes the system.

task do_reset();

This sets the reset signal high and initializes control signals.

Start Counter Task

task start_counter();

Generates a start pulse to begin counting.

Stop Counter Task

task stop_counter();

Stops the counter operation.

5.2 Test Bench Module (counter_tb.sv)

```
`timescale 1ns/1ps

module counter_tb;

parameter WIDTH = 8;

// DUT SIGNALS

logic clk;

logic reset;
```

```
logic start;
logic stop;
logic enable;
logic sel_mode;
logic [WIDTH-1:0] max_value;
logic [WIDTH-1:0] counting_val;
// DUT INSTANTIATION
counter_top #(.WIDTH(WIDTH)) dut (
    .clk(clk),
    .reset(reset),
    .start(start),
    .stop(stop),
    .enable(enable),
    .sel_mode(sel_mode),
    .max_value(max_value),
    .counting_val(counting_val)
);
// CLOCK GENERATION
initial clk = 0;
always #5 clk = ~clk;   // 10ns clock
// TASKS
// Reset task
```

```
task do_reset();
begin
   reset = 1;
   start = 0;
   stop  = 0;
   enable = 0;
   #20;
   reset = 0;
end
endtask
// Start counter
task start_counter();
begin
   enable = 1;
   start  = 1;
   #10;
   start  = 0;   // pulse start (FSM friendly)
end
endtask
// Stop counter
task stop_counter();
begin
```

```
    stop = 1;
    #10;
    stop = 0;
end
endtask
// MONITOR
initial begin
    $display("TIME   COUNT");
    $monitor("%0t    %0d",$time,counting_val);
end
// TEST SEQUENCE
initial begin
    // INITIAL VALUES
    max_value = 8'd10;
    sel_mode  = 1;   // UP MODE
    do_reset();
    // TEST 1 : UP COUNT
    $display("\n--- UP COUNT TEST ---");
    start_counter();
    repeat(15) @(posedge clk);
    // TEST 2 : STOP FUNCTION
    $display("\n--- STOP TEST ---");
```

```
    stop_counter();
    repeat(5) @(posedge clk);
    // TEST 3 : RESUME
    start_counter();
    repeat(10) @(posedge clk);
    // TEST 4 : DOWN COUNT
    $display("\n--- DOWN COUNT TEST ---");
    sel_mode = 0;
    repeat(20) @(posedge clk);
    $display("TEST COMPLETE");
    #20;
    $finish;
end
endmodule
```

---

6. Test Cases

Test Case	Configuration	Expected Result
Up Count	sel_mode = 1	Counter increments
Stop	stop = 1	Counter halts
Resume	start = 1	Counter resumes

Test Case	Configuration	Expected Result
Down Count	sel_mode = 0	Counter decrements

Table 6: Test cases.

---

7. Simulation Results

The simulation waveform shows:

- Proper clock generation
- Reset initialization
- Correct start and stop control
- Up-count and down-count functionality

The counter values observed in the waveform follow the expected behavior.

Example observed sequence:

03 → 04 → 05 → 06 → 07 → 06 → 05 → 04 → 03 → 02 → 01 → 00

This confirms that the counter correctly switches between **up-count and down-count modes**.

The simulation waveform shown in Figure 3 verifies the correct operation of the counter under different modes.

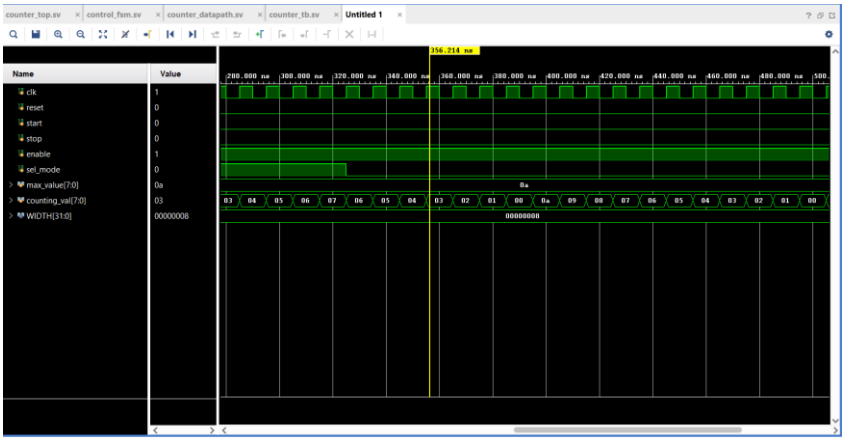

Figure 3: Simulation Waveform Showing Up-Count and Down-Count Operation

The design was verified using a SystemVerilog testbench. The waveform confirms correct FSM transitions and counter functionality.

When the start signal is asserted, the FSM transitions to RUN state and the counter begins counting.

When stop is asserted, the FSM returns to IDLE state and counting stops.

The waveform also verifies correct up-count and down-count behavior depending on the mode selection.

---

8. FPGA Synthesis Results

The design was synthesized using Xilinx Vivado. The synthesis results show efficient utilization of FPGA resources.

The parameterized design allows flexible counter width while maintaining low resource usage.

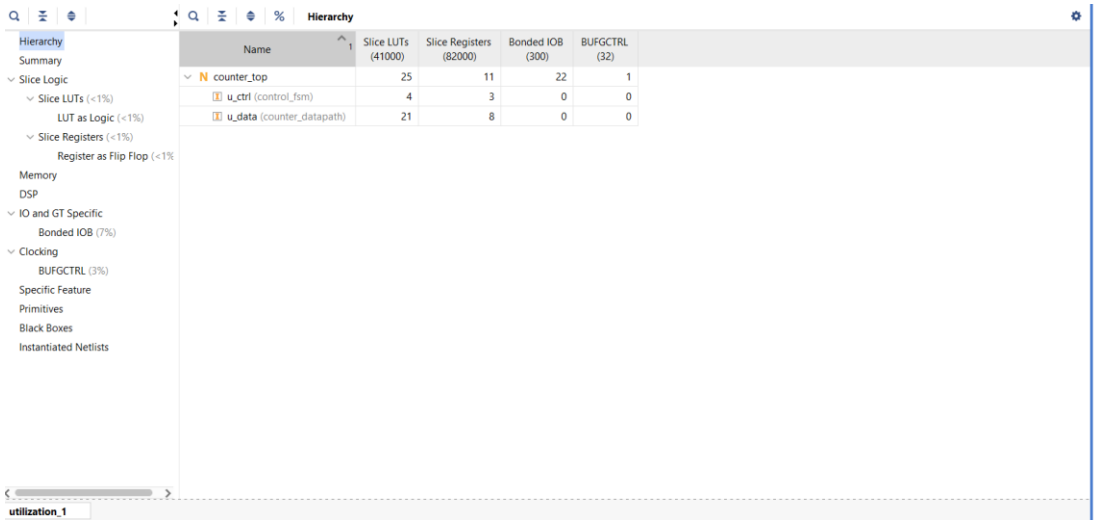

Figure 4: FPGA Resource Utilization Report Generated After Synthesis

After functional verification through simulation, the design was synthesized using Xilinx Vivado.
Synthesis converts the RTL design into a gate-level implementation that can be mapped onto FPGA hardware resources.

The synthesis report provides information about the hardware resources utilized by the design.

Key resource utilization results are summarized below:

Resource	**Utilization**
Slice LUTs	25
Slice Registers	11
Bonded I/O	22
BUFGCTRL	1

Table 7: FPGA Resource Utilization Summary.

The synthesis results show that the design uses a very small portion of the available FPGA resources, demonstrating that the counter design is lightweight and efficient.

---

9. RTL Netlist View

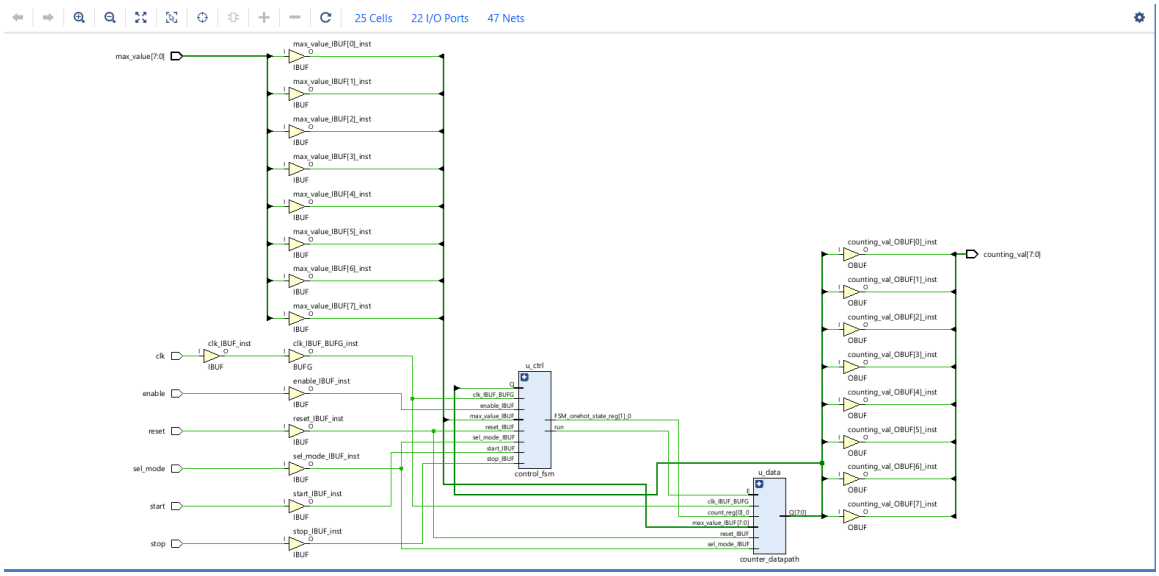

Figure 5: RTL Netlist View of the Synthesized Counter Design

After synthesis, the RTL design can be visualized as a structural hardware representation inside the FPGA.

The RTL netlist diagram illustrates how the modules are connected and implemented using FPGA primitives such as:

- Input buffers (IBUF)
- Output buffers (OBUF)
- Clock buffers (BUFG)
- Logic cells

The diagram clearly shows the integration of the following modules:

- Control FSM
- Counter Datapath
- Input/Output buffers

The Control FSM generates the run signal, which controls the datapath module responsible for the counting operation.

---

10. Timing Analysis

Timing analysis was performed using Vivado Timing Summary after synthesis and implementation.

The design meets timing constraints with positive slack, indicating reliable operation at the specified clock frequency.

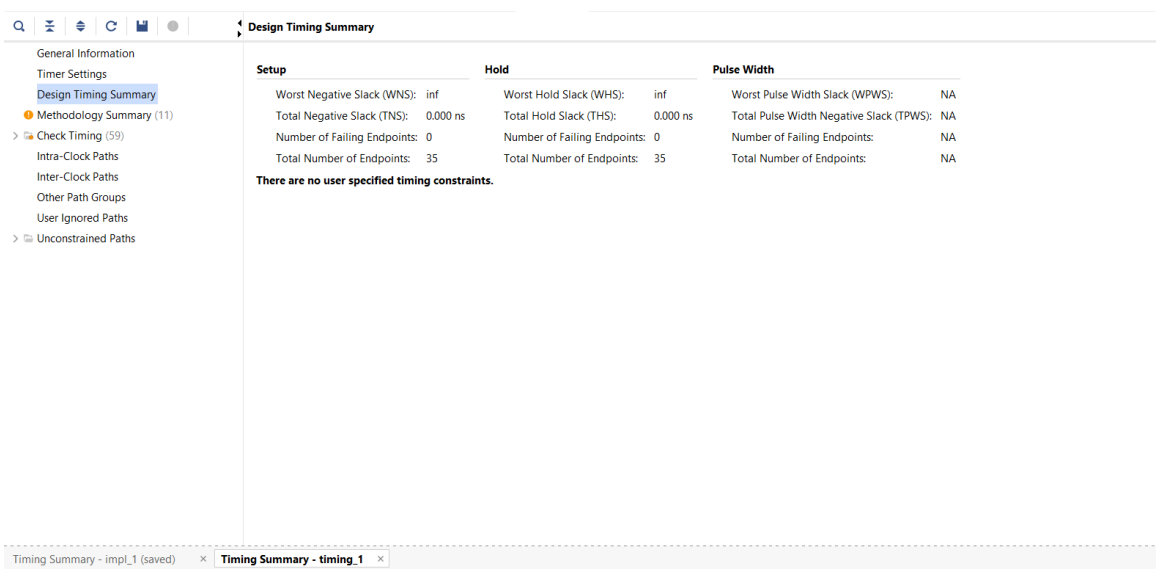

Figure 6: Timing Summary Report Showing No Timing Violations

Timing analysis verifies whether the synthesized design satisfies the required timing constraints of the FPGA.

The design was analyzed with a clock period of 10 ns corresponding to a clock frequency of 100 MHz.

The Vivado timing summary provides important metrics such as:

• Worst Negative Slack (WNS)
• Total Negative Slack (TNS)
• Hold Slack
• Number of failing endpoints

From the generated timing report:

- Worst Negative Slack (WNS): No negative slack observed
- Total Negative Slack (TNS): 0 ns
- Number of failing endpoints: 0

These results indicate that no timing violations are present in the design, and the counter operates correctly within the given clock constraints.

---

11. Applications

The designed counter can be used in several digital systems:

- Digital timers
- Event counters
- Frequency dividers
- Embedded controllers
- FPGA-based digital systems

---

12. Conclusion

This project demonstrated the design and verification of a parameterized counter using a control FSM and datapath architecture in SystemVerilog.

The modular design approach improves scalability and maintainability of the hardware design. The functionality was verified through simulation, and the design was synthesized using Xilinx Vivado.

The project provides practical experience in digital design, FSM modeling, hardware verification, and FPGA synthesis workflows.

---

13. Future Improvements

Future work may include:

- Adding **overflow detection**
- Implementing **programmable counting range**
- Adding **interrupt generation**
- Creating a **UVM-based verification environment**
- Synthesizing the design on an **FPGA board**